CODE REVIEW EVALUATION FORM

JavaScript & Express.js | Undergraduate Programming Course

1. SUBMISSION INFORMATION

Course:	ICS 385	**Section:**	
Instructor:	Debasis Bhattacharya	**Semester:**	Spring 2026
Student Name:	Bryceson Gaoiran	**Student ID:**	28810114
Project Title:	LOS Visitor Data	**Date:**	2/18/2026
Reviewer:	Bryceson Gaoiran	**Review Type:**	Peer / Instructor

2. CODE SUBMISSION DETAILS

Repository URL:	https://github.com/Bryceson-ai/ics385spring2026/blob/main/week6/Hawaii%20Tourism%20LOS%20Calculator/Bryceson_Code_Review_Template_JS_Express_LOS.pdf		
Branch:	Main	**Commit Hash:**	
Files Reviewed:		**Lines of Code:**	

3. CODE OVERVIEW & PURPOSE

Briefly describe the purpose of the submitted code, its main functionality, the Express.js routes implemented, and any middleware or external packages used.

Summary:

This project is a Hawaii tourism Length-of-Stay calculator that imports CSV data into MongoDB and lets users compute statistics by visitor category and optional location through a web interface. It uses Express routes for GET /api/categories, GET /api/locations, POST /api/calculate, and GET /api/data, with middleware/packages including cors, JSON/urlencoded parsers, static file serving, dotenv, and mongoose (plus csv-parser for data import).

4. EVALUATION CRITERIA

Rate each criterion on the scale provided. Use the descriptors as guidance. A score of 4 = Excellent, 3 = Proficient, 2 = Developing, 1 = Beginning, 0 = Not Attempted.

Criterion	Description	Score (0–4)	Weight
Code Correctness & Functionality	Application runs without errors; all Express routes return expected responses; edge cases handled.	3	20%

Criterion	Description	Score (0–4)	Weight
Code Structure & Organization	Logical file/folder structure (e.g., routes/, controllers/, models/); separation of concerns; modular design.	3	15%
Naming Conventions & Readability	Variables, functions, and routes use clear, descriptive names following camelCase conventions; consistent formatting.	4	10%
Express.js Best Practices	Proper use of Router, middleware chaining, error-handling middleware, appropriate HTTP methods and status codes.	3	15%
Error Handling & Validation	Input validation present; try/catch or .catch() used; meaningful error messages returned to client.	4	10%
Comments & Documentation	Inline comments explain non-obvious logic; README or header comments describe setup, dependencies, and usage.	4	10%
Security Considerations	No hardcoded secrets; use of environment variables; input sanitization; helmet or CORS configured if applicable.	3	10%
Testing & Reliability	At least basic test cases provided (e.g., using Jest or Supertest); tests cover primary routes and edge cases.	0	10%

Total Weighted Score:	2.80 / 4.00	**Percentage:**	70 %

5. DETAILED FINDINGS — CODE-LEVEL OBSERVATIONS

Document specific issues, bugs, or noteworthy patterns found during the review. Reference file names and line numbers where applicable.

#	File / Line	Severity	Category	Description / Observation
1	api.js:125-128	High / Med / Low	security/data exposure	Public debug route returns up to 100 database documents without authentication or role checks; this can leak internal data in production.
2	api.js:11	High / Med / Low	error handling /security	API sends raw error.message to clients, which may disclose internal query or runtime details; safer pattern is generic client errors plus server-side logging.
3	server.js:14	High / Med / Low	security	CORS is enabled globally with default allow-all behavior; no origin allowlist is configured for production hardening.
4	api.js:28-40	High / Med / Low	validation	Request validation only checks that category exists; no schema/type validation or sanitization for category/location values before database query usage.
5	importData.js:28	High / Med / Low	data integrity /operations	Import process deletes all existing records before insert; a failed or partial import can leave the collection empty.
6	importData.js:10-11	High / Med / Low	maintain ability /config consistency	Default CSV input is hardwired to standalone/data.csv, which may conflict with documented expected filename/workflow and cause operator confusion.
7	app.js:22-23	High / Med / Low	reliability/UX	Standalone loader reads response text without checking HTTP status first; missing or blocked CSV may surface as parse errors instead of a clear file-not-found message.
8	package.json	High / Med / Low	testing and reliability	No test framework or test script is defined, and no test files were found; primary routes and edge cases are currently unverified by automated tests.

6. EXPRESS.JS & JAVASCRIPT CHECKLIST

Check each item that applies to the submitted code. Mark Y (Yes), N (No), or N/A.

Category	Checklist Item	Y / N / N/A
Server Setup	Server listens on a configurable port (e.g., process.env.PORT)	Y
Server Setup	Entry point file is clearly identified (e.g., app.js or server.js)	Y
Routing	Routes are organized using express.Router()	Y
Routing	RESTful conventions followed (GET, POST, PUT/PATCH, DELETE)	N
Routing	Route parameters and query strings used correctly	N/A
Middleware	Body-parser or express.json() configured for request parsing	Y
Middleware	Custom middleware is reusable and well-documented	N
Middleware	Error-handling middleware defined with (err, req, res, next) signature	N
Async/Await	Promises and async/await used correctly (no unhandled rejections)	Y
Async/Await	Callback patterns avoided in favor of modern async patterns	N
Dependencies	package.json lists all dependencies; no unused packages	Y
Dependencies	node_modules excluded via .gitignore	Y

Category	Checklist Item	Y / N / N/A
Security	Environment variables managed via .env / dotenv	Y
Security	No sensitive data committed to version control	N

7. QUALITATIVE FEEDBACK

Strengths — What does this submission do well?

:

The project shows solid full-stack fundamentals with clear route organization, consistent async/await usage, and a working data pipeline from CSV import to API-backed calculations and frontend visualization.

Areas for Improvement — What should the student focus on next?

:

Focus next on production hardening by adding centralized error middleware, stronger input validation/sanitization, route protection for debug endpoints, and automated API tests.

Suggested Learning Resources

:

Use the official Express.js guide on middleware and error handling together with Joi/Zod validation docs and Supertest examples to build safer, testable route patterns end to end.

8. OVERALL ASSESSMENT

Grade	Range	Description
A / Excellent	90–100%	Code is well-structured, fully functional, secure, and demonstrates mastery of Express.js concepts.
B / Proficient	80–89%	Code works correctly with minor issues; good organization and documentation; some improvements possible.
C / Developing	70–79%	Code runs but has notable gaps in structure, error handling, or best practices; needs revision.
D / Beginning	60–69%	Significant issues with functionality, structure, or documentation; substantial rework required.
F / Incomplete	Below 60%	Code does not compile/run or is largely incomplete; fundamental concepts not demonstrated.

Final Grade Assigned: C		**Numeric Score:**	70 / 100

9. REQUIRED REVISIONS & ACTION ITEMS

List any mandatory changes the student must complete before resubmission.

#	Action Item	Priority	Due Date
1	Add robust request validation for /api/calculate (type/allowed-value checks for category and location) and reject invalid payloads with clear 400 responses.	High / Med / Low	2/20/2026
2	Implement centralized Express error-handling middleware ((err, req, res, next)) and replace direct error.message responses with safe, generic client-facing errors.	High / Med / Low	2/20/2026
3	Protect or remove the debug data endpoint (GET /api/data) so it is not publicly accessible in production (auth check, environment guard, or deletion).	High / Med / Low	2/20/2026
4	Add automated API tests (at minimum: categories, locations, calculate success, calculate invalid input, not-found case) and include a test script in package.json.	High / Med / Low	2/20/2026

10. ACADEMIC INTEGRITY ACKNOWLEDGMENT

By signing below, the reviewer confirms that this evaluation was conducted fairly and objectively. The student acknowledges receipt of this feedback and understands the revisions required.

Reviewer Signature:	Bryceson Gaoiran	**Date:**	2/18/2026
Student Signature:		**Date:**	
Instructor Signature:		**Date:**	